

Цель работы: изучить принципы отображения данных на экранной форме.

Выполнение заданий:

Вариант 11

Задание 1:

1. Осуществите привязку источника данных к элементам управления экранной формы.
2. Разработайте необходимые методы для вывода информации из базы данных на экранную форму.
3. Протестируйте приложение.

Листинг или ссылка на проект:

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace LR26KPIJP
{
    public partial class Form1 : Form
    {
        public Form1()
        {
            InitializeComponent();

            private void toolStrip1_ItemClicked(object sender, ToolStripItemClickedEventArgs e)
            {

            }

            private void label2_Click(object sender, EventArgs e)
            {

            }

            private void textBox3_TextChanged(object sender, EventArgs e)
            {

            }

            private void comboBox2_SelectedIndexChanged(object sender, EventArgs e)
            {

            }

            private void comboBox1_SelectedIndexChanged(object sender, EventArgs e)
            {

            }

            private void Form1_Load(object sender, EventArgs e)
            {
                // TODO: данная строка кода позволяет загрузить данные в таблицу
                "employeeDataSet1.Employee". При необходимости она может быть перемещена или удалена.
                this.employeeTableAdapter1.Fill(this.employeeDataSet1.Employee);
                // TODO: данная строка кода позволяет загрузить данные в таблицу
                "employeeDataSet.Employee". При необходимости она может быть перемещена или удалена.
                this.employeeTableAdapter.Fill(this.employeeDataSet.Employee);
            }
        }
    }
}
```

```

        textBox1.DataBindings.Add("Text", employeeDataSet1, "Employee.Surname");
        textBox2.DataBindings.Add("Text", employeeDataSet1, "Employee.Name");
        textBox3.DataBindings.Add("Text", employeeDataSet1, "Employee.Patronymic");
        textBox7.DataBindings.Add("Text", employeeDataSet1, "Employee.NatName");
    }
    private void textBox1_TextChanged(object sender, EventArgs e)
    {
    }
}

```

Результат работы программы (копия экрана):

Ответы на контрольные вопросы:

1. Как осуществляется подготовка ADO.NET к работе?

Подключение к ADO.NET База данных

Прежде чем работать с базой данных, вы должны добавить (здесь) **OleDb** .NET Data Provider namespace, разместив следующее в начале вашего модуля кода:

```
using System.Data.OleDb;
```

Аналогично для **SqlClient** Пространство имен .NET Data Provider:

```
C#
```

```
using System.Data.SqlClient;
```

using Оператор должен быть первым в вашем коде.

Теперь мы должны объявить строку подключения, указывающую на базу данных MS Access "PersonDatabase.mdb".

```
C#
```

```
public string  
    conString=@"Provider=Microsoft.Jet.OLEDB.4.0;" +  
    @" Data Source=..\..\PersonDatabase.mdb";
```

База данных должна быть в указанном пути, в противном случае вы должны изменить путь соответствующим образом.

Следующий шаг - создать **OleDbConnection** объект. Затем мы передаем строку подключения к этому **OleDbConnection** объект. Теперь вы можете кодировать, чтобы создать новый **ADO.NET** Объект подключения для подключения к базе данных поставщика OLE DB.

```
C#
```

```
OleDbConnection con = new OleDbConnection(conString);
```

Вы также можете явно ссылаться на объявленные объекты, если вы не возражаете печатать много.

```
C#
```

```
System.Data.OleDb.OleDbConnection con = new  
System.Data.OleDb.OleDbConnection(conString);
```

Вот фрагмент кода для подключения к базе данных:

```
C#
```

```
//using declaration for OLE DB  
using System.Data.OleDb;  
//specify the ConnectionString property  
public string conString= @"Provider=Microsoft.Jet.OLEDB.4.0;Data  
Source=..\..\PersonDatabase.mdb";  
//Initializes a new instance of the OleDbConnection  
OleDbConnection con = new OleDbConnection(conString); // open the database connection  
with the property settings  
// specified by the ConnectionString "conString" con.Open();
```

Во многих более ранних приложениях была тенденция открывать соединение при запуске приложения и не закрывать соединение до тех пор, пока приложение не завершится. Это дорогостоящая и трудоемкая операция по открытию и закрытию соединения с базой

данных. Большинство баз данных имеют ограничение на количество одновременных подключений, которые они разрешают.

Например: каждое соединение потребляет определенное количество ресурсов на сервере базы данных, и эти ресурсы **не** бесконечно. Большинство современных поставщиков OLE DB (включая поставщика SQL Server) реализуют *пул соединений*. Если вы создаете соединения с базой данных, они хранятся в пуле. Когда требуется соединение для приложения, поставщик OLE DB извлекает следующее доступное соединение из пула. Когда ваше приложение закрывает соединение, оно возвращается в пул и становится доступным для следующего приложения, которое хочет установить соединение.

Это означает, что открытие и закрытие соединения с базой данных больше не является дорогостоящей операцией. Если вы закрываете соединение, оно **не** означает, что вы отключаетесь от базы данных. Он просто возвращает соединение с пулом. Если вы открываете соединение, это означает, что это просто вопрос получения уже открытого соединения из пула. Рекомендуется во многих **ADO.NET** Книги **не** чтобы сохранить соединения дольше, чем вам нужно. Поэтому вы должны:

- Откройте соединение, когда вам это нужно, и
- Закройте его, как только вы закончите с ним

Например: вот еще один способ получить соединение с базой данных:

C#

```
// setup the global SqlConnection object and constr in your class
private SqlConnection con = null;
private string constr = "Integrated Security=SSPI;" +
    "Initial Catalog=Northwind;" +
    "Data Source=SONY\\MYSQLSERVER;";

private void fnGetConnection() { try
{
    // setup the database connection
    con = new SqlConnection(constr); con.Open(); }catch (Exception ex) {
    MessageBox.Show("Error in connection : "+ex.Message); }finally {
    // dispose of open objects
    if (con != null)
        con.Close(); } //finally
}
```

Например: вы хотите открыть соединение, заполните **DataSet**, и закрыть соединение. Если соединение не удастся, вы хотите получить сообщение об ошибке.

C#

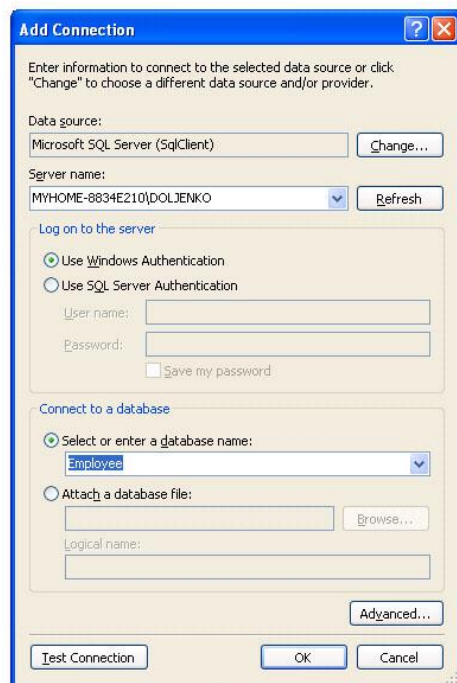
```
try
{
    con.Open(); dadapter.Fill(dataset1); con.Close(); } catch (Exception ex) {
    MessageBox.Show("Error in retrieving data: " + ex.Message); }
```

Например: если вы хотите сохранить измененные данные, вы просто открываете соединение, обновляете данные, закрываете соединение и принимаете изменения. Если это не удастся, отобразите сообщение об ошибке, отклоните изменения и закройте соединение.

C#

```
try
{
    DataSet changes = dataset.GetChanges(); con.Open(); datapter.Update(changes);
con.Close(); dataset1.AcceptChanges(); }catch (Exception ex) {
MessageBox.Show("ErrorR: " + ex.Message); dataset1.RejectChanges(); con.Close(); }
```

2. Как устанавливается связь между БД и приложением?



Выводы: я изучил принципы отображения данных на экранной форме.